

Training: Learning from Mistakes

Loss, gradients, and millions of tiny nudges · video 2:38

This lesson answers the question left open in episode 1: where do the model's numbers come from? A new model starts with **random weights**. Training shows it text, measures how wrong each next-token guess was with one number, the **loss**, and nudges every weight a tiny bit in the direction that makes the loss smaller. Then it does that again, millions of times.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The text is its own answer key	▶ 0:08-0:31	4	☐
2. Cross-entropy: scoring a guess	▶ 0:31-0:57	4	☐
3. Gradient descent: feeling our way downhill	▶ 0:57-1:23	4	☐
4. The learning rate	▶ 1:23-1:31	4	☐
5. Backpropagation	▶ 1:31-1:49	4	☐
6. The training loop in code	▶ 1:49-2:12	4	☐
7. A real loss curve	▶ 2:12-2:27	4	☐

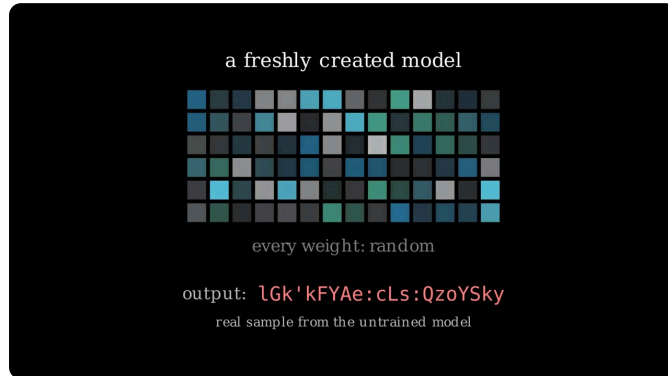
Before you start: this lesson builds on episode 1 (a probability for every token) and episode 10 (logits and softmax). The maths is in Foundations F05 (Exponentials and Logarithms) for the log in the loss, F10 (Slopes and Gradients) for gradient descent, F12 (Neural Networks in Three Minutes) and F13 (PyTorch and Autograd) for backpropagation and the code, and F14 (Train vs Validation Data) for the loss curve.

CONCEPT 1 OF 7

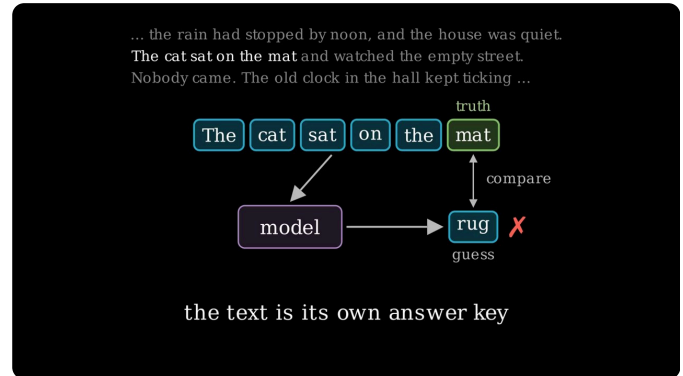
The text is its own answer key

▶ Video 0:08-0:31

A freshly created model is **pure noise**: every weight is random, so it writes gibberish like `lGk'kFYAe:cLs:QzoYSky` (the real untrained tiny GPT). **Training** makes those numbers useful.



▶ 0:16 A new model: random weights, and a real sample of its output.



▶ 0:30 Hide the next token, let the model guess, compare with the truth.

The recipe: take a huge amount of text, **hide the next token**, let the model predict it, and **compare its guess with the truth**. The text itself provides the answers, so nobody labels anything (**self-supervised** learning). Every position is an exercise: from “The” predict *cat*, from “The cat” predict *sat*, ...

KEY IDEA

Training data is just text: at every position the model predicts the next token, and **the text itself says what the right answer was**.

Check 1: The text is its own answer key

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Where do the correct answers for training come from?

- A. People label each sentence by hand
- B. The next token of the text itself
- C. A bigger model grades each guess
- D. A dictionary of correct sentences

1.2 TRUE OR FALSE

“A freshly created model already writes rough English; training only fixes its mistakes.”

True ☐ False ☐

1.3 CALCULATION

“The cat sat on the mat” is 6 tokens. How many next-token exercises does it give?

1.4 SHORT ANSWER

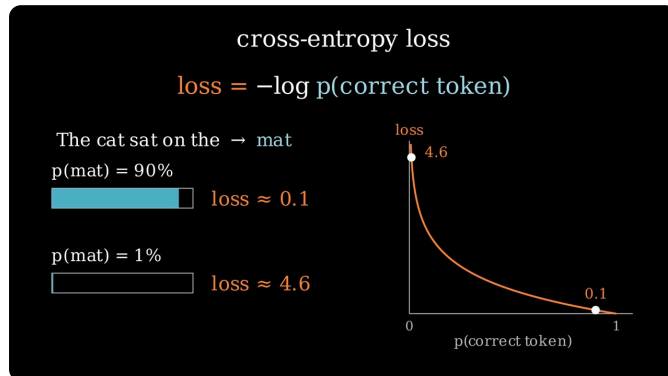
For “To be or not to be”, what are the input and the correct answer of the 4th exercise?

CONCEPT 2 OF 7

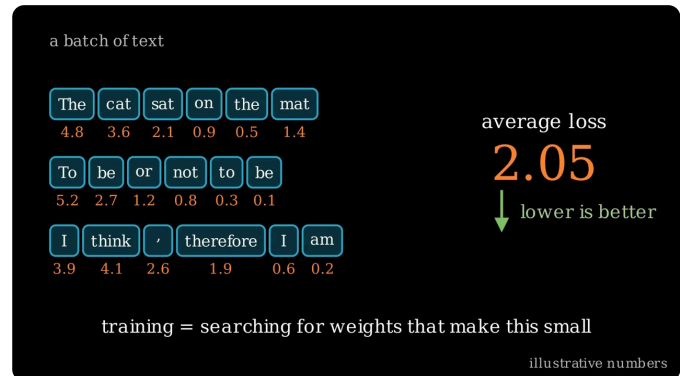
Cross-entropy: scoring a guess

▶ Video 0:31-0:57

To learn, the model needs a number for how wrong it was: the **cross-entropy loss**, $\text{loss} = -\ln p(\text{correct token})$, the negative natural log of the probability it gave to the right token. *mat* at 90 % gives $-\ln 0.9 \approx 0.105$; at 1 %, $-\ln 0.01 \approx 4.605$.



▶ 0:47 90 % for “mat” gives a loss of about 0.1; 1 % gives about 4.6.



▶ 0:56 One loss per position, averaged over the batch (illustrative).

The loss is 0 only for a certain, correct guess ($p = 1$) and grows without limit as $p \rightarrow 0$, so confident mistakes cost a lot. We **average** it over every position in a **batch**. Lower is better: training is the **search for weights that make it small**.

KEY IDEA

loss = $-\ln p(\text{correct token})$, averaged over every position in the batch: near 0 when confident and right, large when the right token got a tiny probability.

Check 2: Cross-entropy: scoring a guess

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

The model gave the correct token *mat* a probability of 0.5. What is the loss?

.....

2.2 MULTIPLE CHOICE

The correct next token is *mat*. Which prediction has the **largest** loss?

- A. $p(\text{mat}) = 0.9$
- B. $p(\text{mat}) = 0.3$, $p(\text{rug}) = 0.6$
- C. $p(\text{mat}) = 0.05$, $p(\text{rug}) = 0.9$
- D. $p(\text{mat}) = 0.5$, $p(\text{rug}) = 0.5$

2.3 TRUE OR FALSE

“A cross-entropy loss can be negative if the model is very sure and right.”

True ☐ False ☐

2.4 CALCULATION

Three positions; the correct tokens got probabilities 0.9, 0.5 and 0.01. What is the average loss?

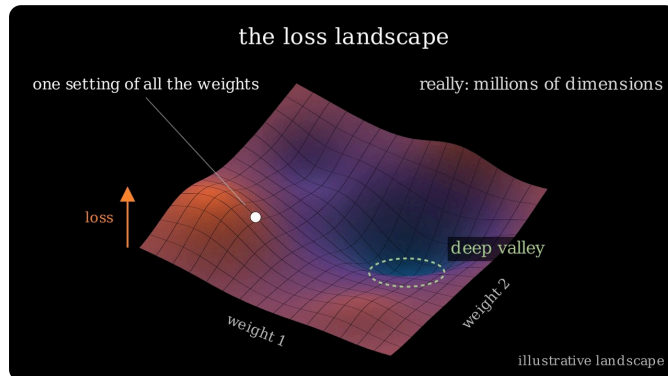
.....

CONCEPT 3 OF 7

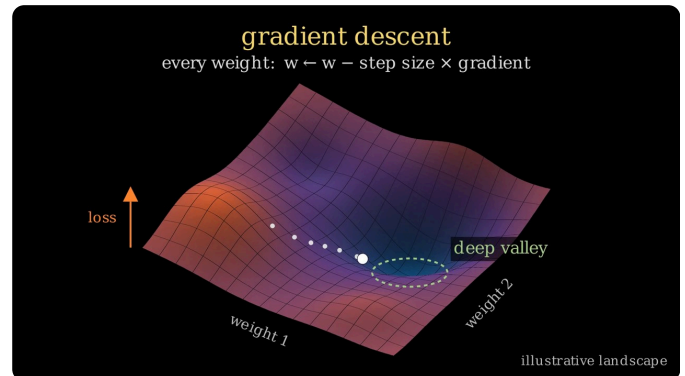
Gradient descent: feeling our way downhill

▶ Video 0:57-1:23

Picture the loss as a **landscape**: each point is one setting of **all** the weights, its height the loss. We want a **deep valley**, but with millions of weights (millions of dimensions) we cannot look around: we feel our way downhill.



▶ 1:09 The loss landscape (illustrative).



▶ 1:22 Gradient descent: step after step, downhill.

The **gradient** gives one slope per weight: which way the loss goes **up**, and how steeply. Every weight steps a little the other way, $w \leftarrow w - \text{step size} \times \text{gradient}$, again and again: **gradient descent**.

KEY IDEA

The gradient points uphill; **gradient descent** steps every weight a little **against** it, over and over.

Check 3: Gradient descent: feeling our way downhill

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

What does one point of the loss landscape stand for, and what does its height mean?

3.2 MULTIPLE CHOICE

For one weight, the gradient is +2.0. What does that tell us?

- A. Set this weight to 2.0
- B. Increasing this weight raises the loss, so decrease it
- C. The loss is 2.0
- D. This weight is twice as important as the others

3.3 CALCULATION

Step size 0.1. New value of (a) $w = 0.50$, gradient +2.0; (b) $w = -1.00$, gradient -3.0?

3.4 TRUE OR FALSE

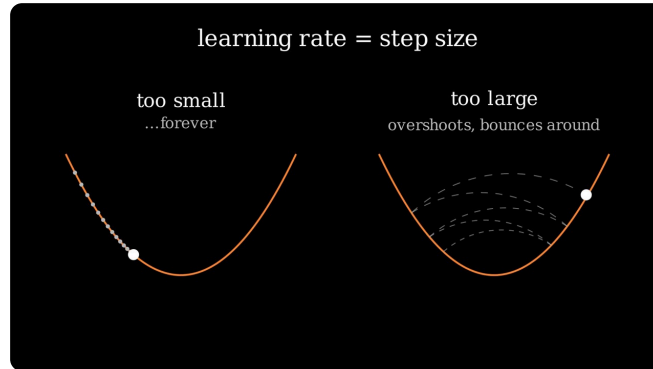
"For a real LLM we could just try every setting of the weights and keep the best."

True ☐ False ☐

The learning rate

▶ Video 1:23-1:31

The size of the step is the **learning rate**. **Too small**, and training takes forever, creeping down the slope. **Too large**, and each step **overshoots** the bottom of the valley, lands on the other side, and training bounces around instead of settling.



▶ 1:30 Too small: tiny steps, forever. Too large: overshoot and bounce.

A toy example: one weight with loss = w^2 , whose slope at w is $2w$. From $w = 1$, a learning rate of 0.1 gives $1 \rightarrow 0.8 \rightarrow 0.64 \rightarrow \dots$, creeping towards the bottom at 0. A learning rate of 1.1 gives $1 \rightarrow -1.2 \rightarrow 1.44 \rightarrow \dots$: every jump crosses the valley and is bigger than the last.

KEY IDEA

The **learning rate** is the step size. Too small: slow. Too large: overshoot and bounce.

Check 4: The learning rate

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

A run's loss goes down steadily, but very slowly. What is the most likely fix?

- A. Lower the learning rate
- B. Raise the learning rate a bit
- C. Train on less text
- D. Stop computing gradients

4.2 TRUE OR FALSE

"If the loss jumps up and down and never settles, the learning rate is too small."

True ☐ False ☐

4.3 CALCULATION

Same toy loss w^2 (slope $2w$), now starting at $w = 3$. Where is w after one step with a learning rate of (a) 0.1 and (b) 1.5?

.....

.....

4.4 CALCULATION

Same toy loss, from $w = 1$, with a learning rate of 0.5. Where is w after one step?

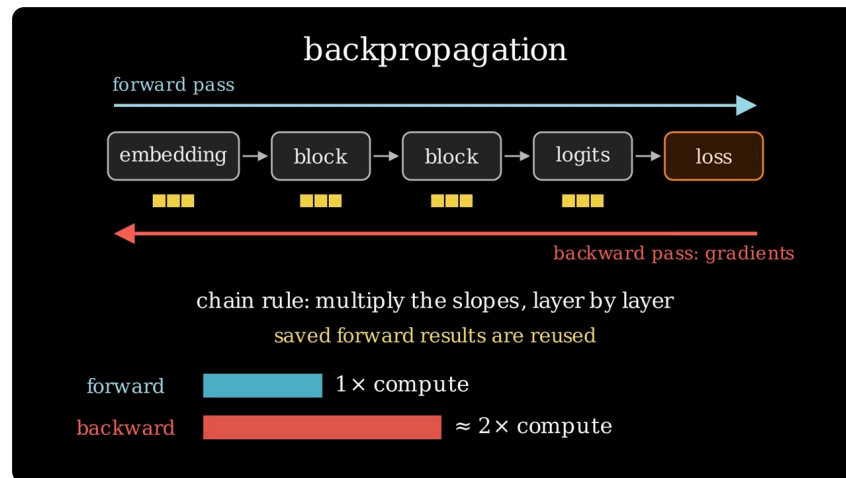
.....

CONCEPT 5 OF 7

Backpropagation

▶ Video 1:31-1:49

Gradient descent needs a gradient for **every** weight, millions of them, at every step. **Backpropagation** gets them all at once.



▶ 1:48 Forward pass left to right; the backward pass carries gradients back from the loss, at about twice the cost.

The **forward pass** runs the model from the embedding through the blocks and logits to the loss, keeping its intermediate results. The **backward pass** then starts at the **loss** and works **backwards** through every layer, applying the **chain rule** from calculus: multiply the slopes, layer by layer. Reusing the saved results, it costs only about **twice** as much as a forward pass.

KEY IDEA

Backpropagation = the chain rule, applied from the loss backwards through every layer. One forward pass plus one backward pass ($\approx 2 \times$ the forward cost) gives the gradient of every weight.

Check 5: Backpropagation

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 PUT IN ORDER

Put these in the order the backward pass works through them: *embedding · logits · loss · blocks*.

5.2 CALCULATION

Chain rule: the loss changes 3 times as fast as a logit, and that logit changes 0.5 times as fast as one weight. What is the loss's gradient for that weight?

5.3 CALCULATION

A forward pass takes 10 ms. Roughly how long does a training step (forward + backward) take?

5.4 TRUE OR FALSE

"Backpropagation finds each weight's gradient by nudging it and re-running the model."

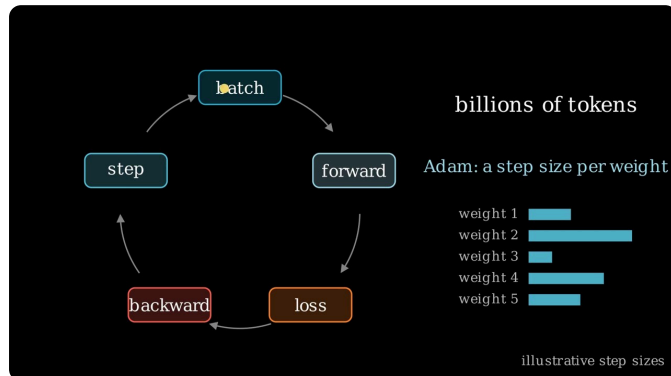
True ☐ False ☐

CONCEPT 6 OF 7

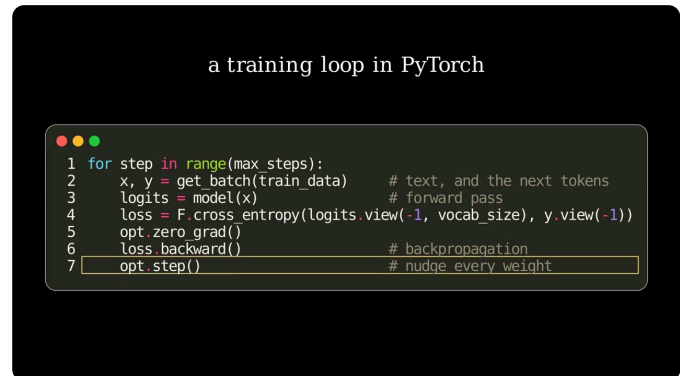
The training loop in code

▶ Video 1:49-2:12

Training repeats one cycle: take a **batch** of text, run the **forward pass**, compute the **loss**, run the **backward pass**, take a small **step**. Over and over, across billions of tokens. In practice the step uses a smarter update rule, **Adam**, which adapts the step size for each weight.



▶ 2:04 The loop: batch, forward, loss, backward, step. Adam gives each weight its own step size (illustrative sizes).



▶ 2:11 The same loop in PyTorch: forward pass, loss, backward pass, step.

```
for step in range(max_steps):
    x, y = get_batch(train_data) # text, and the next tokens
    logits = model(x)           # forward pass
    loss = F.cross_entropy(logits.view(-1, vocab_size), y.view(-1))
    opt.zero_grad()             # backpropagation
    loss.backward()             # nudge every weight
    opt.step()
```

PyTorch does the calculus: `loss.backward()` fills in the gradient of every weight, and `opt.step()` (the optimizer, such as Adam) moves every weight. `opt.zero_grad()` first clears the previous step's gradients, because PyTorch adds new gradients to old ones.

KEY IDEA

One training step = **batch** → **forward** → **loss** → **backward** → **step**. In PyTorch: `model(x)`, `F.cross_entropy`, `loss.backward()`, `opt.step()`.

Check 6: The training loop in code

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 PUT IN ORDER

Put one training step in order: *step · loss · batch · backward pass · forward pass*.

6.2 MULTIPLE CHOICE

What does Adam do differently from plain gradient descent?

- A. It measures the loss in a different way
- B. It adapts the step size for each weight
- C. It no longer needs gradients
- D. It uses a bigger batch of text

6.3 SHORT ANSWER

In the code above, which line does backpropagation, and which line actually changes the weights?

6.4 HANDS-ON CODE

Try it yourself. Here is the same loop for a “model” with a single weight w and the loss $(w - 1)^2$, lowest at $w = 1$. Its gradient is $2(w - 1)$. (a) Before running, predict the three printed lines (step, w , loss, gradient). (b) Delete the line `opt.zero_grad()` and run again. Which numbers change, and why? (c) Put it back and replace `SGD` with `torch.optim.Adam([w], lr=0.1)`. How far does w move in the first step, compared with SGD?

```
import torch

w = torch.tensor(3.0, requires_grad=True)    # one weight, a bad start
opt = torch.optim.SGD([w], lr=0.1)          # plain gradient descent

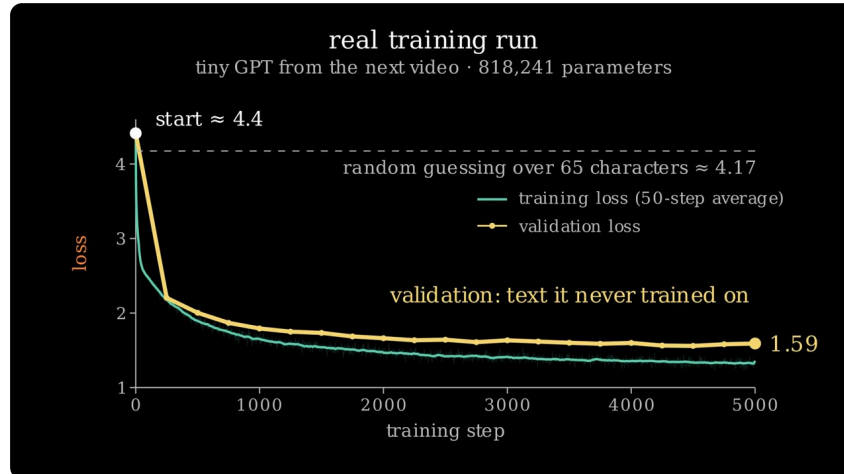
for step in range(3):
    loss = (w - 1.0) ** 2                    # forward pass + loss (lowest at w = 1)
    opt.zero_grad()                          # backpropagation: fills in w.grad
    loss.backward()                          # backpropagation: fills in w.grad
    print(step, round(w.item(), 3), round(loss.item(), 3), round(w.grad.item(), 3))
    opt.step()                              # w <- w - lr * gradient
```

CONCEPT 7 OF 7

A real loss curve

▶ Video 2:12-2:27

This is the real loss curve of the tiny GPT of episode 12, which predicts one of 65 characters at a time. It starts at about **4.4**: no better than guessing. Giving all 65 characters the same probability, $p = 1/65$, gives a loss of $\ln 65 \approx 4.17$ (the dashed line).



▶ 2:26 The real training run of the tiny GPT (818,241 parameters).

It falls **fast at first, then slower**, and ends near **1.59**: the **validation loss**, measured on text the model **never trained on**. It shows what the model learned, not what it memorised.

KEY IDEA

Start ≈ 4.4 (guessing ≈ 4.17), a fast drop, a slow tail, end ≈ 1.59 on text the model never saw: the **validation loss**.

Check 7: A real loss curve

Answer all 4 questions before looking at the answers. Pass mark: all correct.

7.1 CALCULATION

All 65 characters get the same probability. What is the loss of each prediction?

7.2 CALCULATION

The loss went $4.41 \rightarrow 2.20$ in 250 steps, then $\rightarrow 1.59$ by step 5,000. How big is each drop?

7.3 MULTIPLE CHOICE

Why is the validation loss measured on text the model never trained on?

- A. It is faster to compute
- B. To see if what it learned carries over to new text, rather than being memorised
- C. The training text has no correct answers
- D. To make the loss look smaller

7.4 CALCULATION

Which p , given to the right character every time, gives a loss of 1.59? Compare with guessing.

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The text is its own answer key · rewatch ▶ 0:08-0:31

1.1 B. Hide the next token and the text already contains the answer. That is why any text can be used, in huge amounts.

1.2 False. Its weights are random, so it writes random characters (`lGk'kFYAe...`). Everything it knows comes from training.

1.3 5. Every token after the first is a target, predicted from all the tokens before it: cat, sat, on, the, mat.

1.4 Input “To be or not”, answer “to”. The exercises are To → be, To be → or, To be or → not, To be or not → to, ...

2. Cross-entropy: scoring a guess · rewatch ▶ 0:31-0:57

2.1 ≈ 0.69 . $-\ln 0.5 = 0.693$: between the video's 0.1 ($p = 0.9$) and 4.6 ($p = 0.01$).

2.2 C. Only $p(\text{mat})$ counts: $-\ln 0.05 \approx 3.0$, against 0.11, 1.20 and 0.69. How the rest of the probability is spread does not change the loss.

2.3 False. p is at most 1, so $\ln p \leq 0$ and $-\ln p \geq 0$. The best possible loss is exactly 0, at $p = 1$.

2.4 ≈ 1.80 . $(0.105 + 0.693 + 4.605) / 3 = 1.80$. The one bad prediction contributes most of it.

3. Gradient descent: feeling our way downhill · rewatch ▶ 0:57-1:23

3.1 One setting of all the weights; its height is the loss with those weights. A real model's landscape has one direction per weight: millions of dimensions.

3.2 B. The sign says which way is uphill, the size how steep. We step the other way.

3.3 (a) 0.30. (b) -0.70 . $w - 0.1 \times \text{gradient}$: $0.50 - 0.20 = 0.30$ and $-1.00 + 0.30 = -0.70$. A negative gradient means the weight goes up.

3.4 False. With millions of weights there are far too many settings. We only feel the slope where we stand, and step downhill.

4. The learning rate · rewatch ▶ 1:23-1:31

4.1 B. Steady but slow progress is the sign of steps that are too small.

4.2 False. That is the sign of a learning rate that is too *large*: every step overshoots the valley.

4.3 (a) 2.4. (b) -6 . The slope at 3 is 6. (a) $3 - 0.1 \times 6 = 2.4$, a bit closer to 0. (b) $3 - 1.5 \times 6 = -6$: across the valley and further from the bottom than before.

4.4 0. $1 - 0.5 \times 2 = 0$, exactly the bottom. On this toy bowl 0.5 is a perfect step size; real landscapes are never this simple.

5. Backpropagation · rewatch ► 1:31-1:49

5.1 loss → logits → blocks → embedding. It starts where the forward pass ended, at the loss, and works back towards the input.

5.2 1.5. Multiply the slopes along the chain: $3 \times 0.5 = 1.5$.

5.3 About 30 ms. The backward pass costs about twice the forward pass: $10 + 20 = 30$ ms. The weight update itself is cheap.

5.4 False. That would need one run per weight, millions per step. Backpropagation gets every gradient from a single backward pass.

6. The training loop in code · rewatch ► 1:49-2:12

6.1 batch → forward pass → loss → backward pass → step. The loss needs the forward pass, the gradients need the loss, and the step needs the gradients.

6.2 B. It still follows the gradients, but each weight gets its own step size.

6.3 `loss.backward()`, then `opt.step()`. `backward()` only computes gradients; the weights move in `opt.step()`.

6.4 (a) 0 3.0 4.0 4.0, 1 2.6 2.56 3.2, 2 2.28 1.638 2.56 · (b) the gradients become 7.2 and 8.96 · (c) 0.1 instead of 0.4 (a) At $w = 3$ the gradient is $2 \times 2 = 4$, so w moves $0.1 \times 4 = 0.4$ to 2.6; then 0.32 to 2.28. The steps shrink as the slope flattens. (b) Without `zero_grad` the gradients pile up: $4 + 3.2 = 7.2$, then $7.2 + 1.76 = 8.96$, so the steps are too big (w jumps to 1.88). (c) w becomes 2.9: Adam's first step is about the learning rate, 0.1, however steep the slope, because it scales each weight's step by that weight's own gradients.

7. A real loss curve · rewatch ► 2:12-2:27

7.1 ≈ 4.17 . $p = 1/65 \approx 0.015$ and $-\ln(1/65) = \ln 65 \approx 4.17$: the dashed “random guessing” line.

7.2 2.21, then only 0.61. Fast at first, then slower: the easy patterns are learned first.

7.3 B. A model could score well on text it has memorised. New text shows what it really learned.

7.4 $p \approx 0.20$ (1 in 5). $-\ln p = 1.59$, so $p = e^{-1.59} \approx 0.20$, against $1/65 \approx 0.015$ for guessing: about 13 times better.